



AI 同程 *ai code*

1 0 2 4 技术大会

2024.10.24 苏州 | 北京 | 成都

高可用架构设计现代范式

苗文志

Viken

城市通研发部负责人&架构师

何为架构师？



1. 识别出挑战性的问题；
2. 把简单的问题变复杂；
3. 把复杂的问题变简单；
4. 把这个过程变成经验。

Why HA?

- **交付能力的需要：**交付能力（质量+效率）
- **业务发展的需要：**万丈高楼平地起，系统可用性是业务发展的基石，可用性无法保障，好用就无从谈起，更别说技术驱动产品力跃升了。
- **个人成长的需要：**高可用设计是软件架构设计的核心内容之一，在个人的工作、求职、晋升中，其设计能力代表着我们的岗位胜任力和竞争力。缺少高可用架构设计能力，很难走到技术的深水区。



特点：范围广、碎片化。在6升7段位，相关问题出现的概率很高，但回答的并不好。

挑战：规模化的业务增长，给系统稳定性提出了新要求，碎片化的经验明显不足以构建可靠的高可用架构，需要把经验进阶到系统化的方法论，才能有更好的落地性。

XX事业群生产事故定级及处罚规定

2.事故级别:

类别	P5	P4	P3	P2	P1	P0
一级业务损单	无损单	0%<	>0.5%	>1%	>5%	>10%
二级业务损单	无损单	0%<	>1%	>5%	>10%	>15%
促前利润损失	---	<=5万元	>5万元	>10万元	>50万元	>100万元
完全停止响应	---	<5分钟	<10分钟	<30分钟	<1小时	>1小时
全站流量损失	---	<1%	<3%	<5%	<15%	>15%
核心数据丢损	---	<1%	<3%	<5%	<10%	>10%
核心数据延迟	---	<5分钟	<10分钟	<30分钟	<1小时	>1小时
损单故障等级提升	---	P5 且>1H	P5且>2H; P4且>1H;	---	---	---

- 事故报告样本
 - 事故影响：影响的产品线或项目
 - 事故时间：事故持续的时长
 - 影响损失：描述具体量化影响，单量、资损；用户视角的体验。
 - 处理过程：事故发生和处理过程回放
 - 原因分析：事故的直接原因+根本原因
 - 改进计划：临时解决方案+最终解决方案，事故预防和改进计划
- 线索
 - 事故时长（缩短故障时长和频次）
 - 事故影响访问（缩小影响范围-核心到核心、全部到局部、控制资损）

业界经常用N个9作为衡量系统可用性的指标：

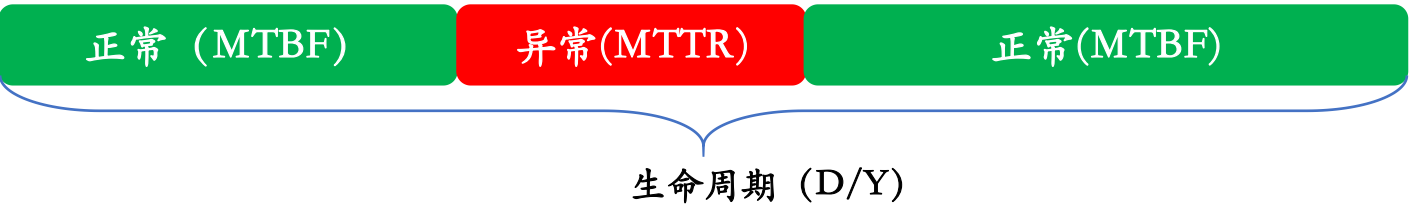
Availability指标：

$$\frac{\sum \text{正常时长}}{\sum \text{正常时长} + \sum \text{异常时长}} \times 100 \%$$

Level of Availability	Percent of Uptime	Downtime Per Year	Downtime Per Day	Reference Range
1 Nine	90%	36.5 d	2.4 h	无可用性
2 Nines	99%	3.65 d	14 m	基本可用
3 Nines	99.9%	8.76 h	86 s	较高可用
4 Nines	99.99%	52.6 m	8.6 s	高可用
5 Nines	99.999%	5.25 m	0.86 s	极高可用
6 Nines	99.9999%	31.5 s	8.6 ms	神级高可用

N的参考值：

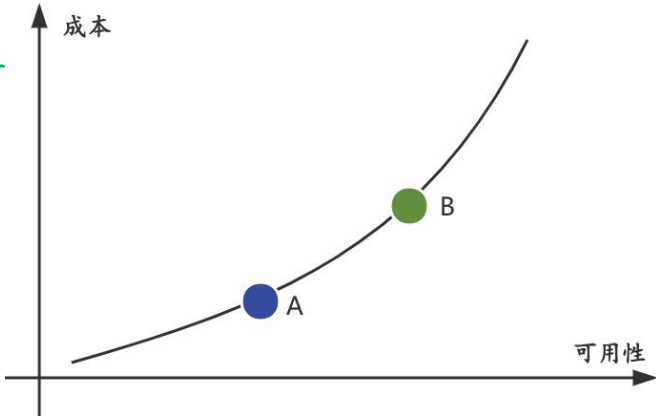
- 1-无可用性
- 2-基本可用
- 3-较高可用
- 4-高可用（承诺），具备自动恢复的高可用能力（努力降低人工干预的操作）
- 5-极高可用性（目标）



首先，解决故障最好的方法是不发生故障。其次，真发生了，恢复的时间越短越好

总结下来，高可用架构设计最终发力点在这一长一短的两件事情上：

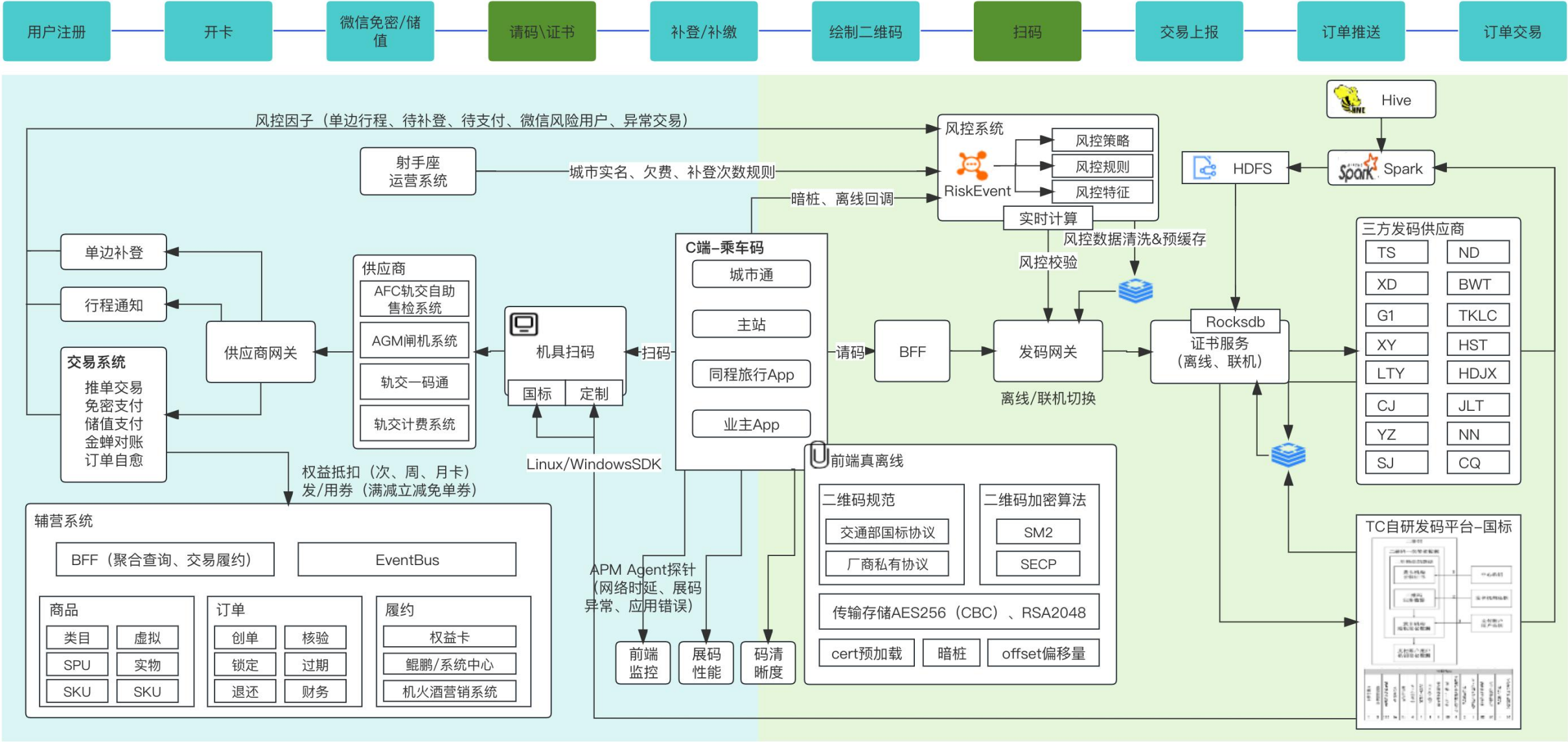
- 1、让MTBF更长
- 2、让MTTR更短（降低频次、缩短时长、保障核心流程可用）



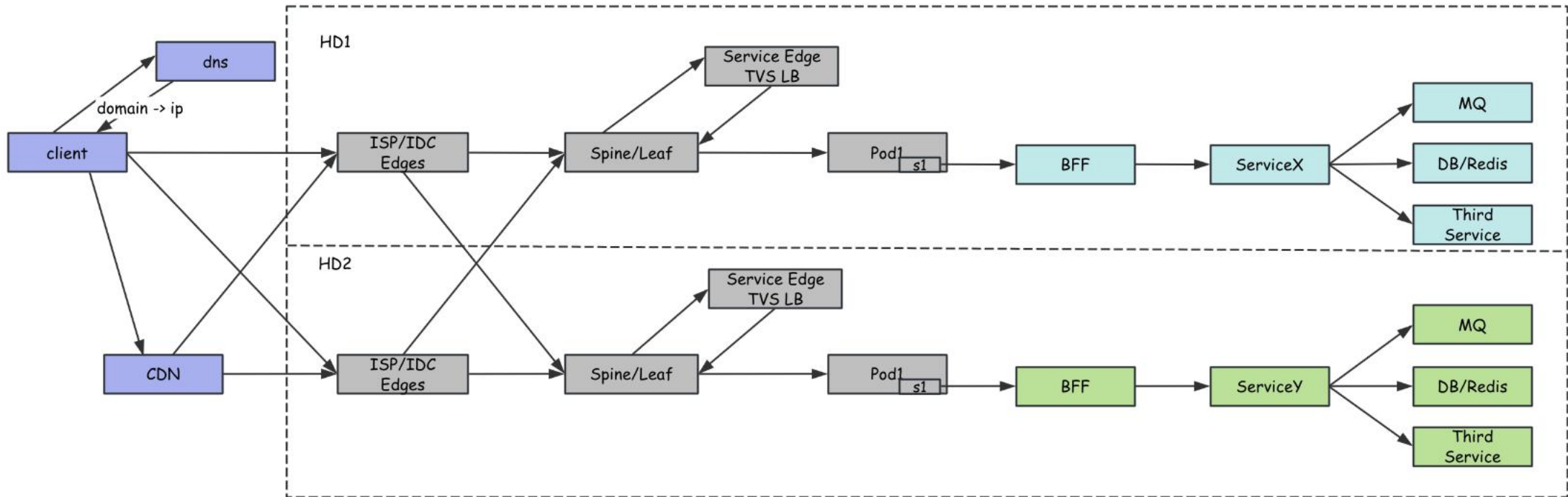
让MTBF更长

洞察问题，设计升维，见叶知秋、未雨绸缪

城市通乘车码业务流程概况



思考：用户扫码乘车场景下，会有哪些可用性的问题？



方法：FMEA（Failure Mode and Effects Analysis）

1. 把业务流程抽象转化成架构设计草图

2. 假设法，穷举某个过程或依赖出现不可用的可能性

3. 分析出现不可用对系统功能会产生的影响的严重性，是否会影响到用户使用

4. 不可用导致的问题是否能够被测试和监控到

5. 根据不可用发生的概率及影响程度，评估优化的必要性，设计优化方法
- 联网前：地铁进站用户无网、腾讯云ISP故障

• 网络异常：偏远地区网络延时高、DNS劫持

• 接入层：早晚高峰和节假日，峰值流量难以预测，随时会有突发流量出现，首页如城市列表等资源请求量突然增长，对带宽和接口产生压力；单一城市的流量突然增长，对系统产生压力，容易连锁反应，对其他城市产生影响。

• 逻辑层：供应商服务能力参差不齐，三方接口性能或网络抖动，产生超时或不可用，对系统和用户产生影响；

• 中间件：MQ、DB、Redis等中间件故障不可用导致接口异常，服务故障；

• 机房级：太湖机房异常或机房专线网络异常，数据无法正常同步，导致数据出现不一致性。

1. 用户使用场景下无网、弱网。

期望：小程序乘车码页面能正常打开。加载出依赖资源：

定位、城市列表、供应商信息

底导和menu_tab

乘车码QR

策略：上下文弱依赖+冗余+离线化处理

底导和menu_tab压缩后打入主包，更新咋办？

延迟更新策略

资源更新加速？充分利用CDN、Quic

离线码

2. ISP故障，区域接入网络中断

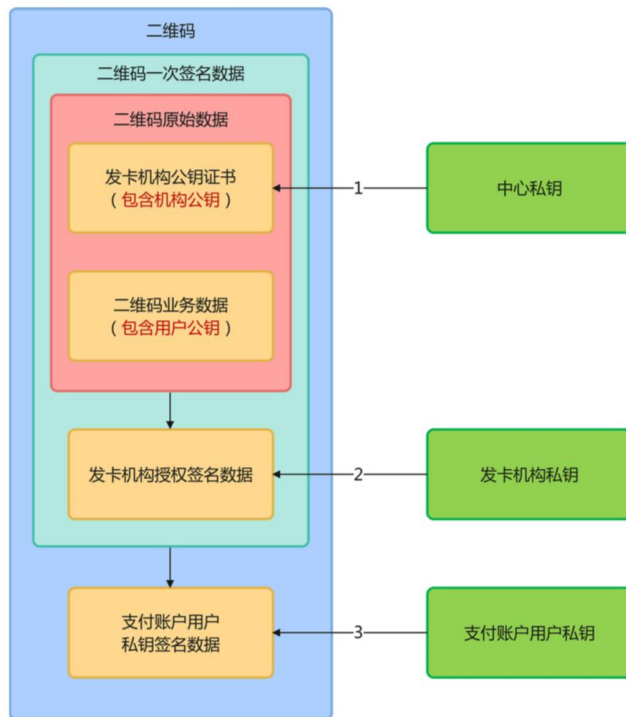
主备域名容灾设计

域名测速

域名切换

复位

3. DNS解析异常或劫持



总结：

网络方面：

DNS，避免localDNS异常导致的网络劫持或接口请求失败问题。

内容校验、HttpDNS、异常监控。

CDN，资源加速，链路加速（主干网络优化）、削峰、降本、冗余。

混合CDN，主动更新（push）和被动回源（pull）合理使用。

应用方面：

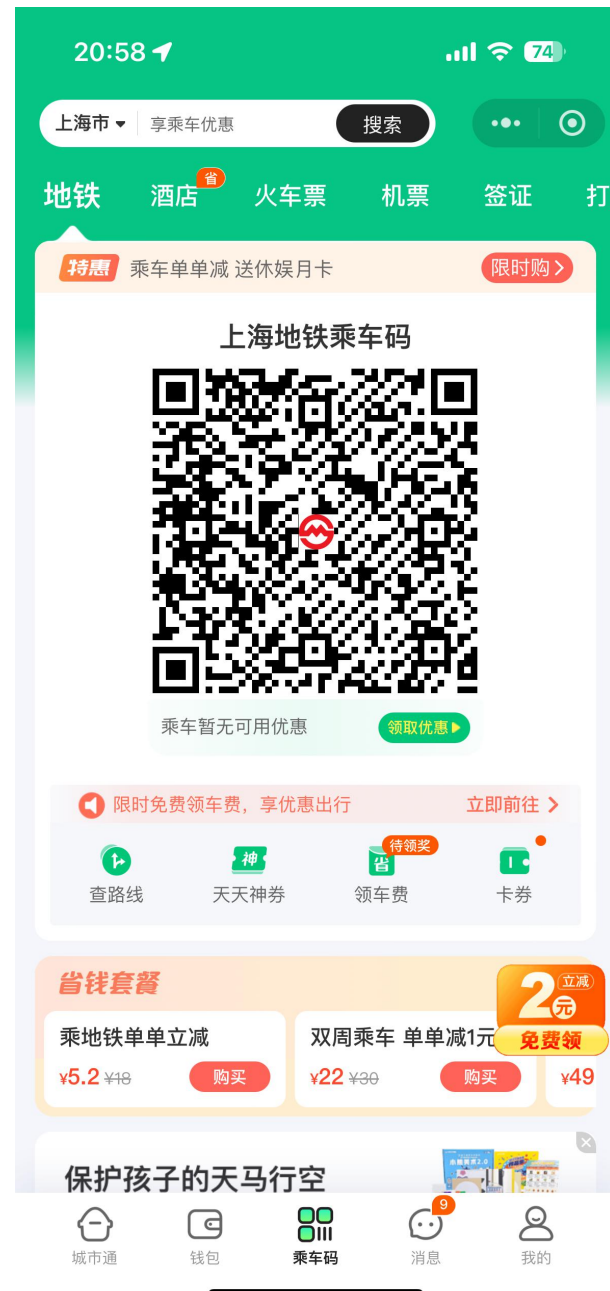
弱依赖，后端的依赖？上线文的依赖？

强缓存

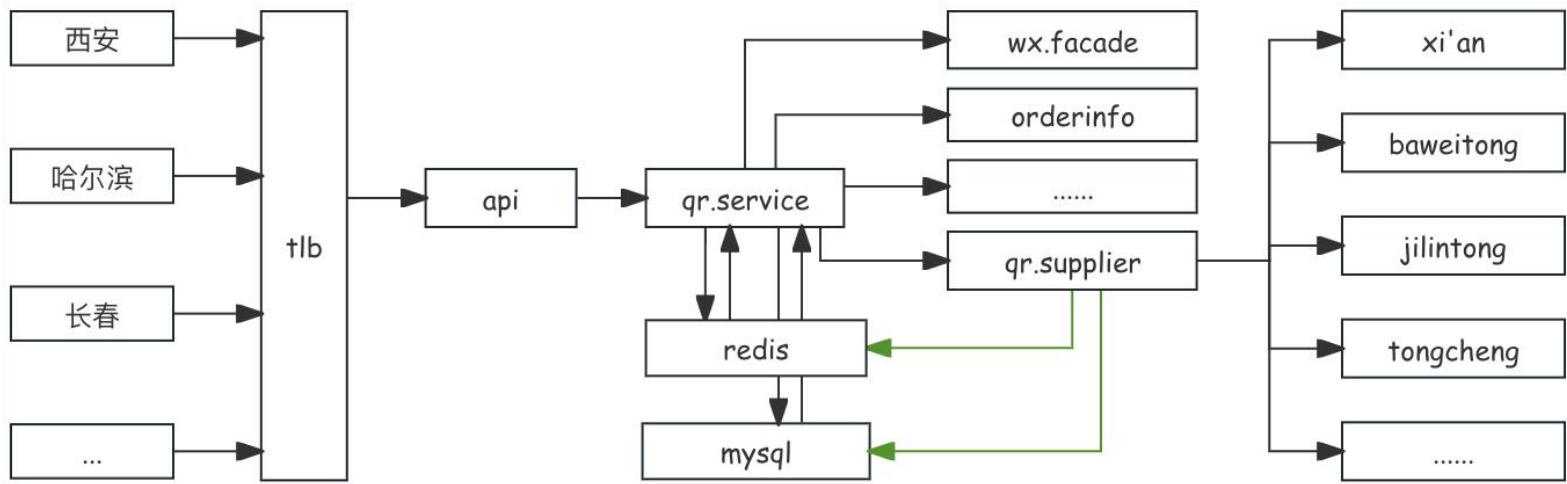
延迟更新

冗余（数据、兜底逻辑）

监控



早期的请码/证书接口架构



说明：

- 目标：100%成功率
- 亮点：
 - 无状态：具备水平扩缩容的能力。
- 不足：
 - 代码耦合，大单体，每次代码变更和发版影响全局。
 - 运行时耦合，单一供应商故障容易引起所有城市集群性能波动
 - 超时时间默认5、4、3s，供应商单一节点异常，导致线程资源无法及时释放
 - 用户是否能展码依赖多个服务，链路长，处理时间长，还容易出错。
 - 对redis强依赖，redis故障，mysql扛不住，引发雪崩。
 - 扩展性不足，随着用户量增长，redis内存膨胀，容量和读写性能会成为瓶颈

优化思路：

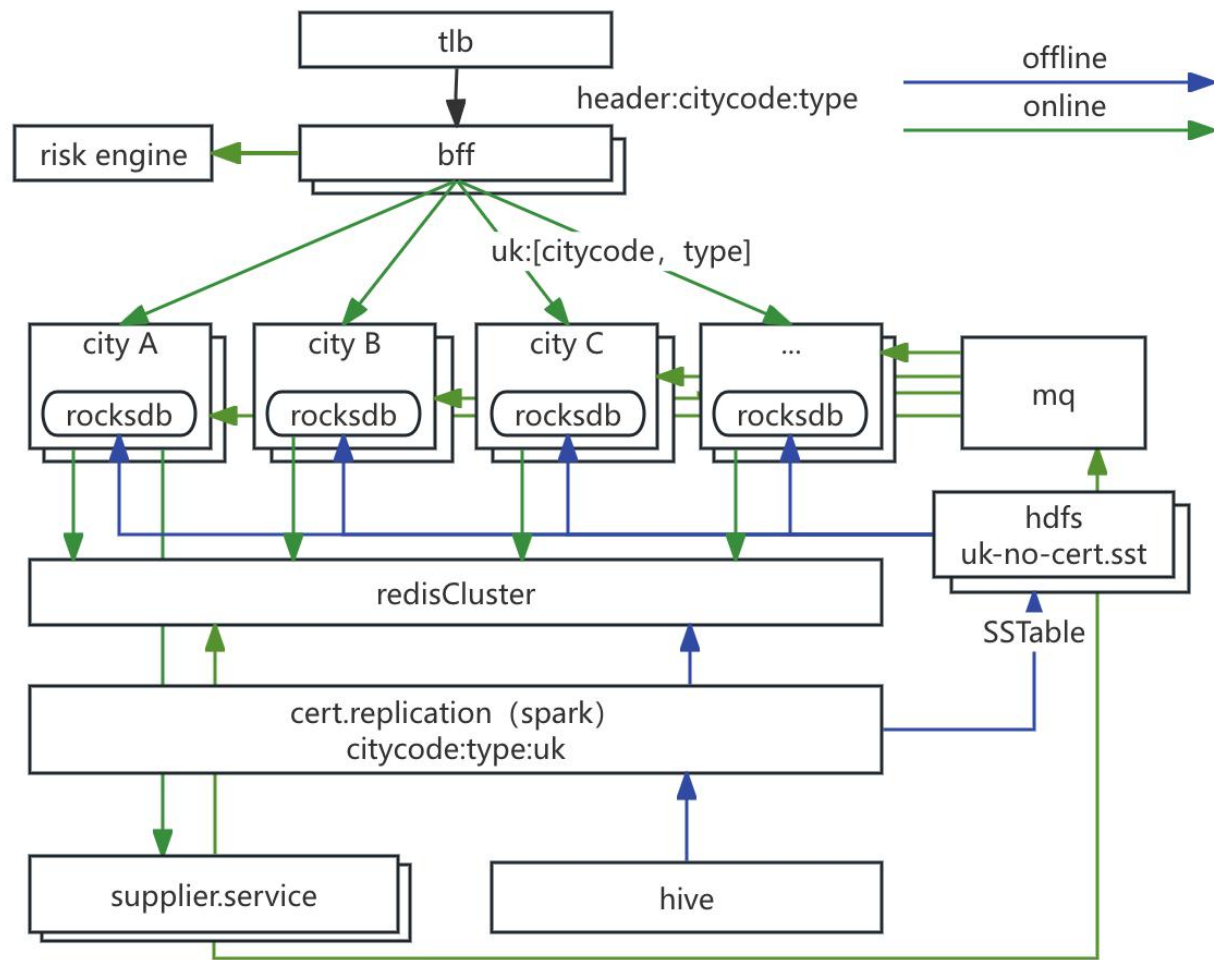
- 无状态：无上线文数据依赖
- 隔离问题（解耦）
 - 领域解耦，依赖倒置
 - 代码隔离
 - 运行时隔离，应用解耦
 - 集群隔离，城市互不影响
 - 数据隔离
- 扩展性
 - 订单量从百万到千万，打破QPS上限
- 冗余（最小依赖）
 - 缓存前移
 - 日志，用来告警、fix问题或异常恢复
- 防呆设计
 - 过滤，校验，补偿，切换，兜底。
- 超时控制
 - 三方接口的超时时间需要更加精细化、动态化的控制
- 重试&幂等
 - 下游重试，上游幂等；避免重试风暴
- 过载保护
 - 限流、熔断、降级
 - EDA（异步化、MQ）
- 恢复能力
 - 手动，柔性+补偿+幂等设计，发生异常后，系统和交易自动恢复。
 - 自动

不破不立，打破思维定势，合理选择技术实现方案，解决实际问题

hd1/2

qr

code/uk



领域解耦

- 用户展码风险和状态以事件形式接入风控系统
- 前置的风控校验从证书核心服务中抽离，在bff层聚合。

隔离

- 公共的代码被封装成sdk，差异化的代码通过策略模板隔离。所有代码在同一个仓库，确保能复用，差异隔离，同时满足职责单一和开闭原则。
- 运行时隔离，不同的城市新增独立UK做独立集群，逻辑变更、版本发布、供应商异常的风险完全隔离。

最小依赖

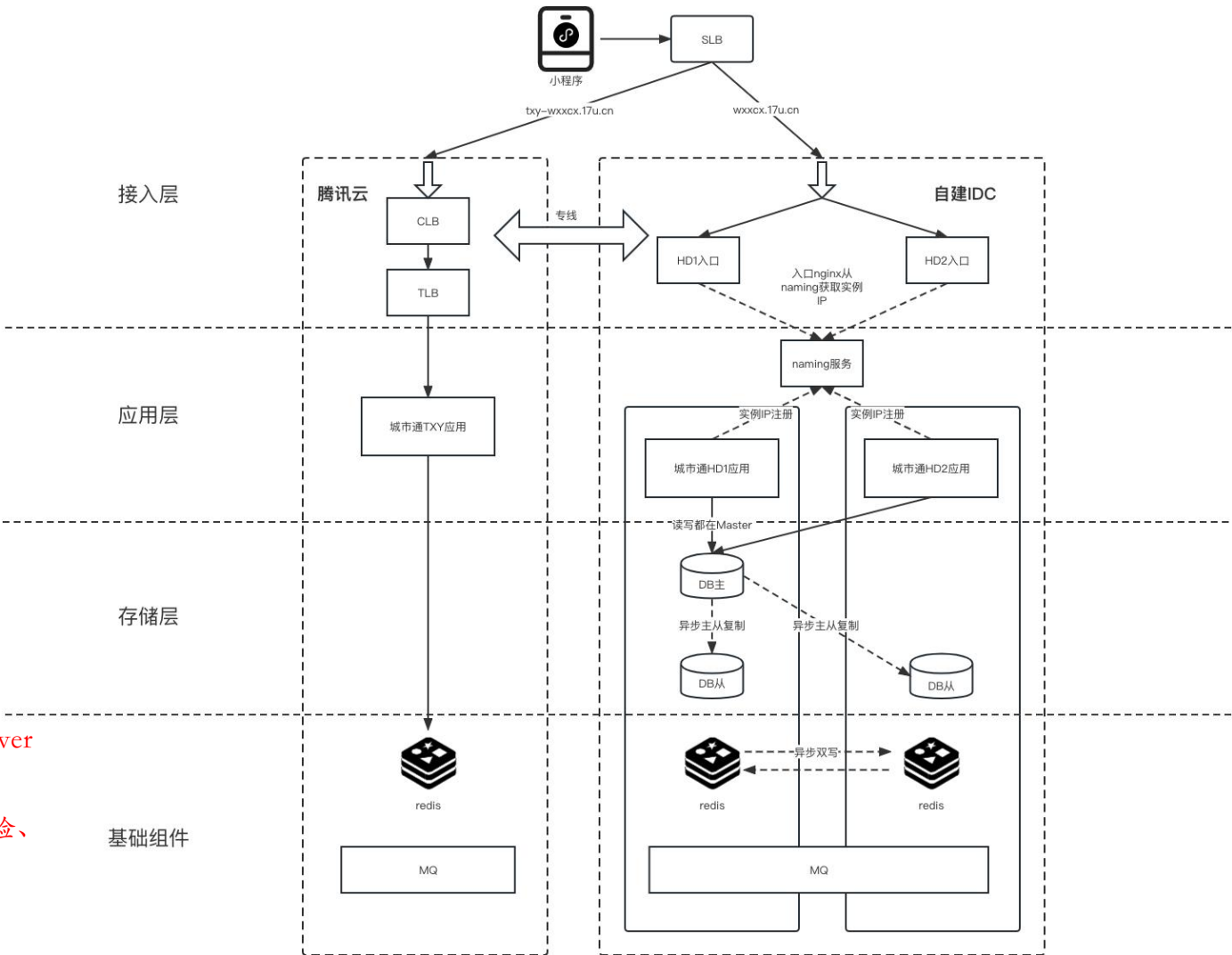
- rocksdb+mq,对redis实现了冗余的同时，提升了性能上限和容量上限。
- 高峰期通过mq的增量同步，和非高峰期，通过离线sstable方式进行全量更新的组合方式，避免了rocksdb的写放大问题。
- redis集群从强依赖下降为弱依赖

超时控制

- failfast原则，Default->TP90*2->EMA*2，快速失败，释放线程资源，避免蝴蝶效应造成系统雪崩

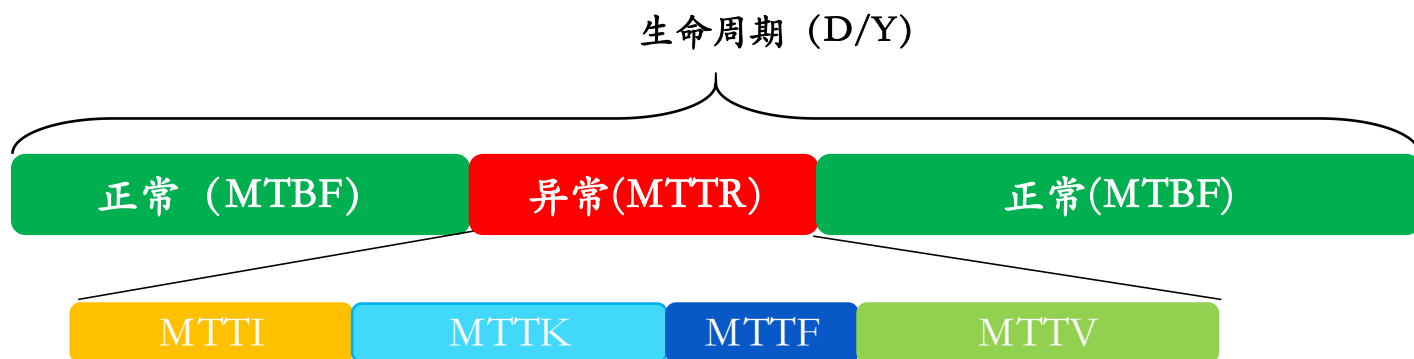
积极响应、合理利用

前端	无/弱网、离线码、APM探针
网络	CDN, DNS优化, 强缓存、Quic加速、域名容灾
机房	异地多活 (双中心)、公有云容灾单元 (成本)
接入层	TEFE, Naming, Drax流量监控/错误监控、神盾局
应用	系统容错性设计、防呆设计、failfast原则、过载保护、最小依赖、分层/分治、有界、自动恢复
中间件	DB、Config、Cache、MQ
CI/CD	弹性扩容、潮汐扩容, 优雅下负载, 优雅关机, Failover
监控预警	系统监控、应用监控、前端apm、微信告警、日志巡检、AIOps、夜莺监控、神盾局、天网 业务监控: 同环比指标异常预警
故障演练	故障预案&演练: 每周一次故障演练 (网络、应用、中间件、IDC故障)、应急方案、处理流程



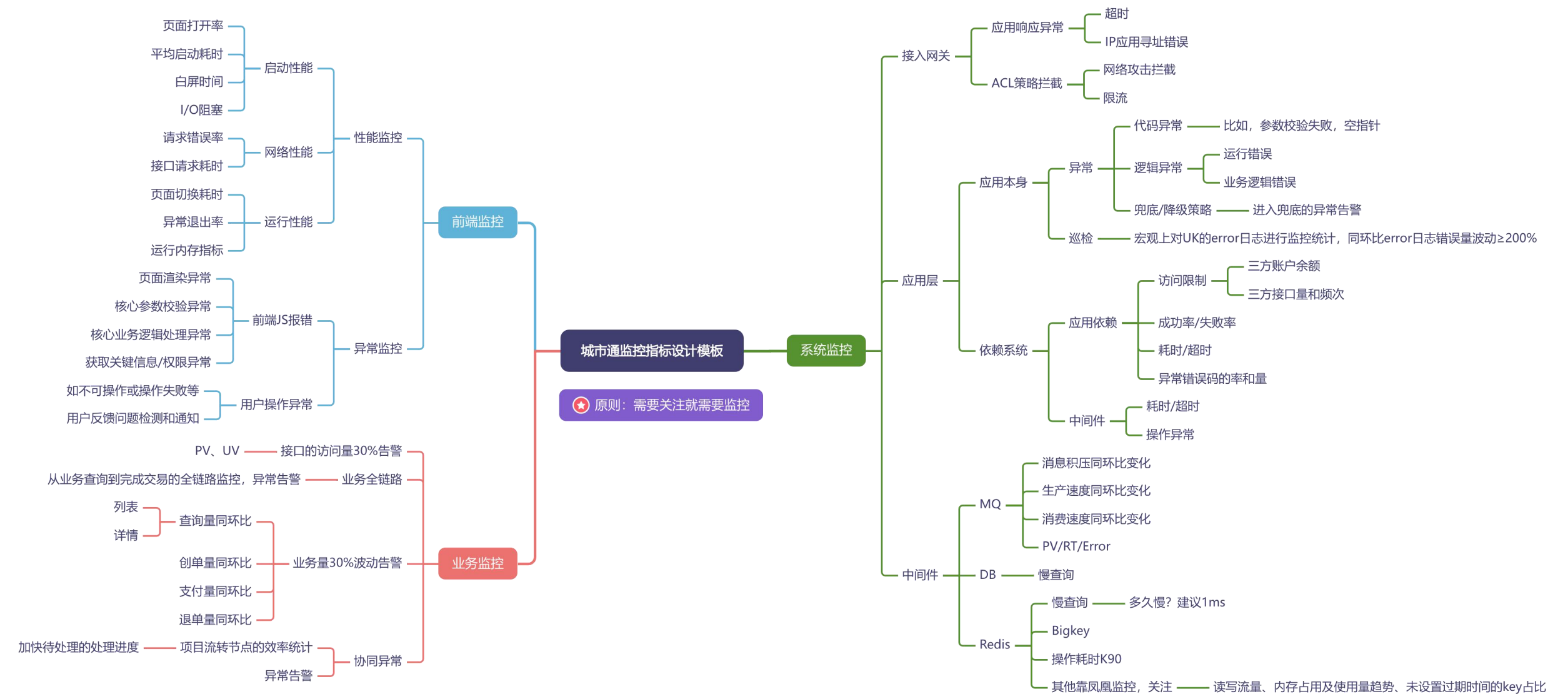
让MTTR更短

搭建全局指标体系，让系统可观测（可感、可控、可恢复）

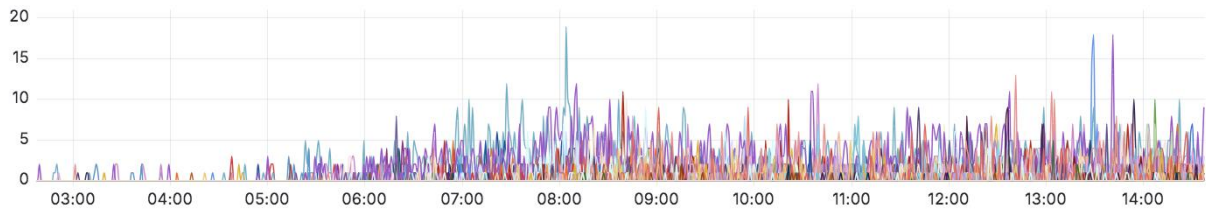


MTTR: Mean Time To Repair, 故障平均修复时长

- MTTI (Mean Time To Identify, 平均故障发现时长), **可感**能力搭建, 系统化的监控 (监控指标设计模板、日志打印和Metric同步规范)、巡检能力搭建 (人工、系统)、值班SOP规范。
- MTTK (Mean Time To Know, 平均故障定位时长), **可控**能力搭建, 充分利用devops一站式开发运维平台, 通过对runtime的指标分析, 快速定位到问题根因, 建议日常把JVM指标纳入系统巡检。
- MTTF (Mean Time To Fix, 平均故障修复时长), **可恢复**能力搭建, 也就是从知道了根因在哪里, 到我们采取措施恢复业务为止。优先回滚、限流、降级、熔断、重启, 其次修复上线。
- MTTV (Mean Time To Verify, 平均故障修复验证时长), 可验证性, 测试用例的完整性、测试脚本、自动化测试, 灰度验证。

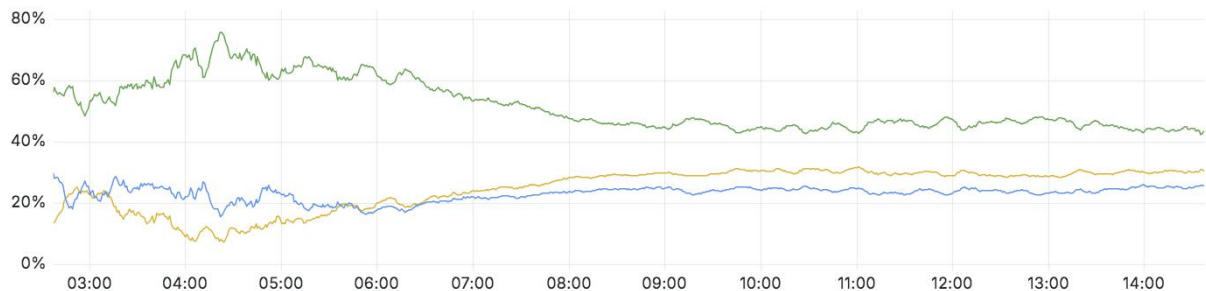


风控异常code(1m)



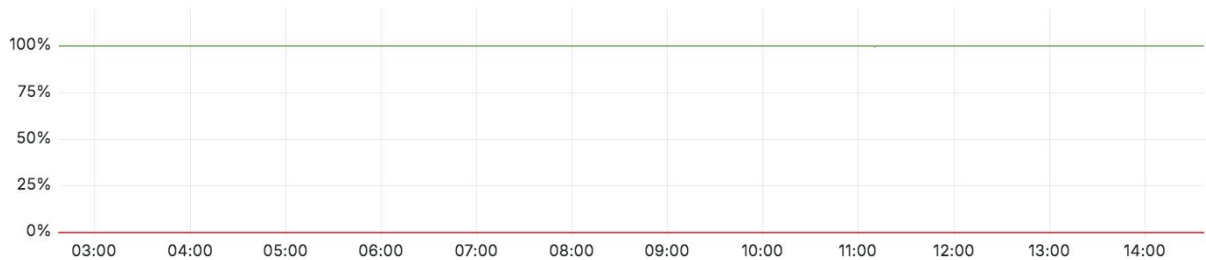
— {cityCode="130100", message="已注销用户请码风控"}
— {cityCode="130100", message="当前用户欠款笔数达到上限，待补缴"}
— {cityCode="130100", message="用户存在异常出站情况，需补登"}

证书数据命中率(10m)



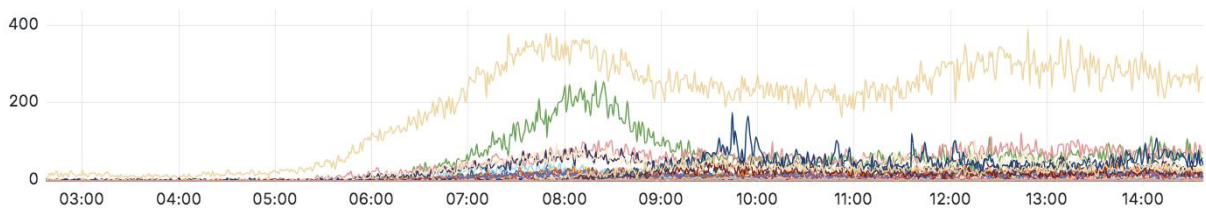
— Rocksdb命中率 Mean: 51.9% — 供应商命中率 Mean: 24.8% — Redis命中率 Mean: 23.2%

Rocksdb请求成功率



— 请求成功 — 异常java.io.FileNotFoundException — 异常java.lang.IllegalArgumentException — 异常java.util.concurrent.TimeoutException
— 异常org.apache.hadoop.ipc.RemoteException

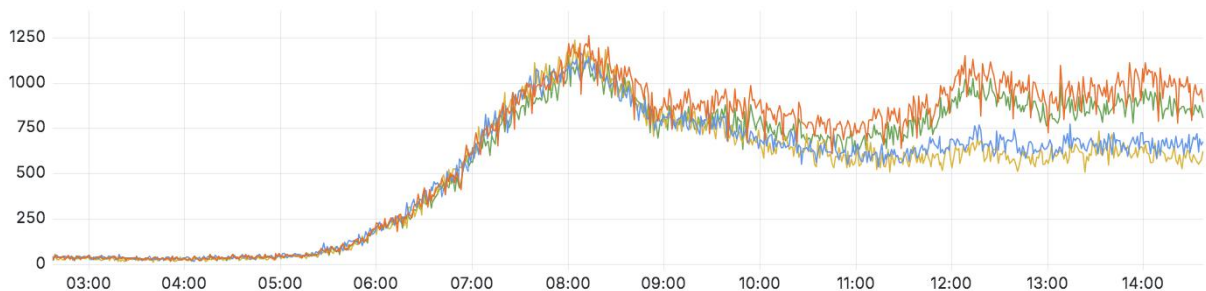
证书接口请求量-城市维度(1m)



— {cityCode="130100"}
— {cityCode="130181"}
— {cityCode="130200"}

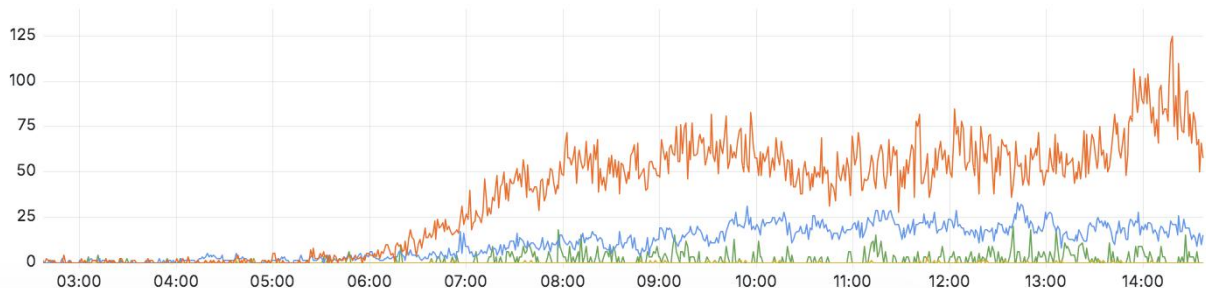
Mean
56.7
0.0291
0

证书接口请求量(1m)



— 证书服务-今天 — 证书服务-昨天 — 证书服务-上周 — 网关-getQrCert

离线码服务日志数量(1m)



— 证书服务-error日志 — 证书服务-warn日志 — 网关-error日志 — 网关-warn日志

从经验到方法论，再到流程机制，让经验可以复制，最终达到“功成不必在我”。

1、**设计升维原则**，思考问题要尽可能全面，解决问题可以降维，阶段性实施。

2、**最小依赖原则**（强依赖->弱依赖->无依赖）

方法：FMEA、有罪推论，识别风险，评估影响，降低依赖）

3、**隔离原则**，控制故障范围的有效手段，尤其是系统已初具规模流量。

根据核心和非核心、访问规模等维度，尝试做代码隔离、线程隔离、进程隔离、读写隔离、数据隔离。

4、**分层原则**

解耦，领域、结构、代码清晰，可以有效降低混乱导致的bug出现概率，提升可维护性。

5、**冗余原则**

通过对物理设施、网络、应用、容器、存储、缓存等冗余设计，降低单点故障对服务连续性和稳定性的影响。

6、**过载保护原则**，或手动或自动通过限流、熔断、降级等手段，保障核心服务可用，把影响降到最低。

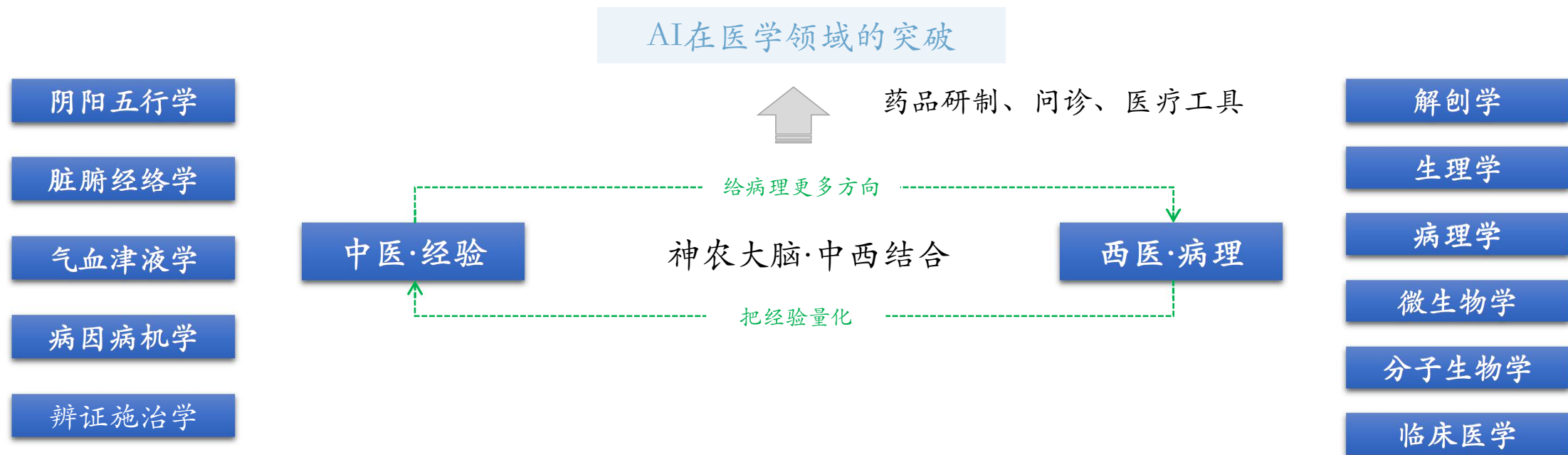
7、**可观测原则**，可感（发现问题）、可控（诊断问题）、可恢复（恢复问题）。

8、**平衡原则**，高可用每多一个动作，都会增加系统研发投入和资源的成本，比如超时重试、单元化。

未来范式

AI，已逐步成为“新质生产力”的核心推手

经验从被学习到被打破，被重塑，才是能力上升螺旋的开始。



样本-经验-改进

代码分析：代码语法问题-代码逻辑问题-代码可用性方面问题-性能问题-代码运行环境问题-数据存储问题。

异常处理：重在先知先觉，AI擅长更细维度的异常预判、捕获和处理。

自学习，自迭代：AI比人学习经验的能力更强，更全面，对物理设备性能更了解，两者结合在快和稳上实现成本和可用性的“动态平衡”。

THANK YOU